INSTITUTO DE EDUCACIÓN SUPERIOR TECNOLÓGICO PÚBLICO “CHINCHA”

IESTP “Chincha” · Chincha, Ica, Perú

Manual técnico del sistema

Arquitectura, instalación, mantenimiento y recuperación

PORTAL INSTITUCIONAL E INTRANET

10 de noviembre de 2026 Portal institucional e intranet https://iestpchincha.edu.pe

Documentación para administración de sistemas, soporte y desarrollo. Describe la versión de código revisada y sus procedimientos de operación; los datos y credenciales del alojamiento se mantienen en el inventario privado.

Institución	Ubicación
IESTP Chincha	Av. Unión N.° 500, Pueblo Nuevo, Chincha, Ica.

Alcance y uso del manual

Este documento explica cómo instalar, mantener y recuperar el sistema del IESTP Chincha. Se dirige al personal técnico responsable de PHP, base de datos, alojamiento, tareas programadas y control de cambios.

Referencia de la edición

Elemento	Referencia
Dirección institucional	https://iestpchincha.edu.pe/
Código examinado	ranyuo/chincha · commit 14e4488271c321a9403b7a8b4f96cc5baa928ae7
Esquemas comprobados	Repositorio 5; respuesta del sitio publicado 7. No se certifica equivalencia.
Fecha	10 de noviembre de 2026.
Comprobaciones de esta edición	Sintaxis, instalación y navegación local; acceso publicado, con contraste de versiones en anexo C.

Cómo interpretar las instrucciones

Las rutas /ruta y /home/<usuario> son plantillas: deben sustituirse por los valores reales del alojamiento. Las órdenes se muestran para ejecución por personal autorizado en el entorno correspondiente. No son una afirmación de que se hayan aplicado al servidor institucional.

Los ejemplos no contienen contraseñas reales. Los valores sensibles se completan en archivos privados o mediante los mecanismos del proveedor. Las configuraciones de desarrollo, QA y producción deben mantenerse separadas.

Límites de la revisión

Se inspeccionaron código y documentación y se ejecutaron pruebas locales. No se inspeccionó la configuración interna del hosting de iestpchincha.edu.pe, ni se realizó un despliegue o restauración en producción. Confirme versiones y rutas del servidor antes de aplicar un procedimiento.

DOCUMENTACIÓN COMPLEMENTARIA

El Manual del sistema explica el uso por módulos. El Cuaderno de talleres organiza las prácticas de capacitación. Este manual técnico documenta componentes, configuración y operación del servicio.

Índice técnico · Arquitectura e instalación

Contenido	Página
01. Arquitectura y flujo general	5
02. Plataforma y requisitos	6
03. Distribución de directorios	7
04. Mapa del código y dependencias	8
05. Preparación del despliegue	9
06. Base de datos y cuenta de servicio	10
07. Configuración privada y secretos	11
08. Instalación inicial	12
09. Servidor web, HTTPS y dominio	13
10. Desarrollo local y separación de entornos	14
11. Modelo de persistencia	15
12. Diccionario: estado y cuentas	16
13. Diccionario: archivos y operación	17
14. Estado JSON y contratos funcionales	18
15. Versiones y compatibilidad de esquema	19
16. Entrada HTTP y enrutamiento	20
17. Sesión, CSRF y autenticación	21
18. API de contenido y administración	22

Las entradas enlazan a cada capítulo. En Word puede abrir el panel de navegación; en PDF puede utilizar los marcadores y la búsqueda de texto.

Índice técnico · Operación y recuperación

Contenido	Página
19. Formularios públicos e idempotencia	23
20. Adjuntos, validación y descarga	24
21. Concurrencia, revisiones y parches	25
22. Permisos, cuentas y alcance	26
23. Integridad académica y documentos	27
24. Correo notificador y configuración	28
25. Notificaciones y seguimiento de entrega	29
26. Cron y tareas de mantenimiento	30
27. Respaldo: alcance y protección	31
28. Restauración portable	32
29. Exportación SQL y traslado de hosting	33
30. Actualización y recuperación de una versión	34
31. Auditoría, errores y observabilidad	35
32. Capacidad y rendimiento	36
33. Controles de seguridad y mantenimiento	37
34. Comprobaciones y pruebas de aceptación	38
35. Diagnóstico por síntomas	39
36. Rutinas de operación y continuidad	40
Anexo A. Configuración y comprobaciones iniciales	41
Anexo B. Rutas y respuestas HTTP	42
Anexo C. Evidencia de pruebas	43
Anexo D. Inventario y control de entrega	44

01. Arquitectura y flujo general

El portal integra web pública e intranet sobre una aplicación PHP. El navegador presenta la interfaz y genera documentos; el servidor valida permisos, guarda información y atiende las solicitudes. La base de datos es la fuente de persistencia compartida.

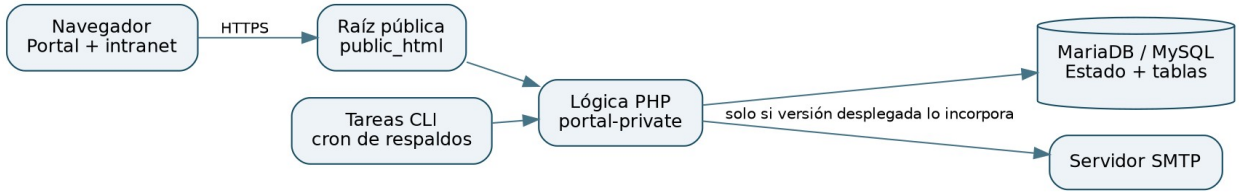


Figura 1. Componentes de ejecución y dependencias principales.

Capa	Responsabilidad
Navegador	Interfaz, formularios, navegación por fragmentos y generación de PDF/Excel.
public_html	Entrada HTTP, archivos estáticos y páginas auxiliares autenticadas.
portal-private	Reglas, sesión, transacciones, contenido y respaldos.
MariaDB / MySQL	Estado institucional, cuentas, archivos, auditoría, ajustes y visitas.

Las tareas programadas usan la misma lógica privada mediante PHP CLI. No existe un servicio Node.js obligatorio para publicar la aplicación. La caída del SMTP no debe confundirse con una pérdida del trámite: el registro y el aviso tienen ciclos separados.

Este esquema describe el código revisado. La topología real del hosting, el motor de base de datos, la versión PHP activa y las políticas de red deben confirmarse en la administración del servidor.

02. Plataforma y requisitos

El destino documentado es un alojamiento PHP con una base MariaDB o MySQL por sitio. El prefijo de tablas pisco_ es fijo; no se documenta un parámetro para sustituirlo.

Requisito	Base de referencia
PHP	8.2 o superior. Verifique versión web y CLI por separado.
Base de datos	MariaDB 10.4+ o MySQL 8; InnoDB y utf8mb4.
Extensiones	pdo_mysql, fileinfo, openssl, zlib, json y session.
Extensión recomendada	mbstring.
Servidor web	Configuración compatible con las directivas Apache del proyecto.
Ubicación	Raíz de dominio o subdominio, no una subcarpeta.
Escritura	Directorio privado y public_html/assets/cache/.
Desarrollo y QA	Docker para la instalación aislada; Node y Chromium para revisar la navegación.

Comprobación previa

```
php -v
php -m
php -r 'echo ini_get("upload_max_filesize"), PHP_EOL;
 echo ini_get("post_max_size"), PHP_EOL;'
```

La salida de PHP CLI no demuestra qué versión utiliza el servidor web. Confirme ambas desde el panel del hosting. Revise límites de subida, memoria, tiempos de ejecución, tamaño de petición y capacidad MySQL de acuerdo con los archivos realmente admitidos.

03. Distribución de directorios

La separación entre raíz pública y código privado es parte del despliegue. Copie ambas carpetas al mismo nivel y configure el document root exclusivamente sobre public_html.

```
directorio-del-dominio/
  portal-private/
    config.php                # privado, no versionado
    schema.sql
    data/portal.json
    views/portal.php
    backups/                  # copias portables
    cron.php
  public_html/
    index.php
    install.php               # retirar tras instalar
    assets/app/
    assets/vendor/
    assets/chincha/
    assets/cache/
```

PORTAL_CONFIG_FILE permite seleccionar una configuración privada en la instalación local documentada. Los componentes de correo no forman parte del repositorio de esquema 5; sus rutas deben identificarse en el despliegue que los incorpore.

No publique copias ZIP del proyecto, SQL, registros ni configuración dentro de la raíz web. Las directivas de denegación complementan la separación física; no la reemplazan. Conserve los nombres y permisos de los archivos operativos al actualizar.

04. Mapa del código y dependencias

PortalApp coordina conexión, sesión y operaciones. El inventario siguiente corresponde al repositorio de Chincha revisado; no presupone que todos los archivos del sitio publicado coincidan con esa revisión.

Archivo o grupo	Función técnica
Portal.php / bootstrap.php	Aplicación, conexión PDO, sesión, entrada y respuestas.
Schema.php / schema.sql	Esquema 5 y creación de tablas.
Policy.php / Accounts.php	Permisos, perfiles, vínculos y gestión de cuentas.
Patch.php / Rules.php	Cambios parciales y validación de contenido.
Academic.php / Domain.php	Coherencia académica y operaciones de dominio.
Forms.php / Services.php	Prueba antispam y servicios públicos.
Files.php	Carga, validación, almacenamiento y acceso a adjuntos.
Backups.php / Audit.php	Copias portables e historial de operaciones.

La interfaz reside en public_html/assets/app. Las bibliotecas del navegador se distribuyen localmente en assets/vendor y sus orígenes se documentan en FUENTES.md. No se necesita Node ni Composer para servir esta versión.

El repositorio no incluye Mail.php, Notifications.php, SmtpSettings.php ni mail-worker.php. Si el sitio publicado ofrece funciones de correo, documente sus archivos desplegados antes de modificarlas. No copie estos componentes desde otro instituto sin una actualización completa y probada.

Antes de cambiar una regla identifique la validación del servidor y el control de permisos asociado. La presencia de un botón en JavaScript no autoriza la operación.

05. Preparación del despliegue

Prepare la publicación como un cambio identificable, con un punto de recuperación y verificaciones posteriores. No utilice una cuenta o base de otro instituto para completar valores que falten.

1. Identifique dominio, directorio del sitio, versión del código y responsable del cambio.

2. Confirme acceso al panel, gestión de base de datos, PHP y tareas cron.

3. Obtenga una copia de contenido y cuentas y resguarde por separado configuración y clave de instalación.

4. Compruebe capacidad de disco, escritura privada y límites de subida y MySQL.

5. Prepare y valide la versión en una instalación de pruebas.

6. Defina ventana, pruebas de humo y criterios para recuperar el servicio.

Dato que debe registrarse	Dónde obtenerlo
Document root y ruta privada	Configuración del sitio en el hosting.
Versión PHP web / CLI	Panel y comando php -v de la ruta programada.
Base, usuario y servidor	Administración de bases del alojamiento.
Correo y restricciones de salida	Proveedor SMTP y panel del hosting.

Este manual no contiene credenciales reales. Los valores de conexión y las rutas del hosting se completan en el entorno autorizado. El dominio institucional usado en los ejemplos es https://iestpchincha.edu.pe/.

06. Base de datos y cuenta de servicio

Cree una base exclusiva para CHINCHA y un usuario de aplicación limitado a esa base. La aplicación necesita ejecutar las consultas de trabajo y los cambios de esquema previstos por la instalación y las actualizaciones.

1. Cree base y usuario desde el panel del alojamiento.

2. Asocie el usuario con la base y revise los permisos disponibles.

3. Confirme servidor, puerto y nombre completo, incluidos los prefijos que imponga el hosting.

4. Configure db_host, db_port, db_name, db_user y db_password en el archivo privado.

5. Pruebe la instalación en una base nueva o la conexión de la instalación existente sin volver a ejecutar una instalación sobre datos de producción.

```
-- Consultas de inventario, solo lectura
SELECT VERSION() AS motor;
SELECT DATABASE() AS base_actual;
SHOW TABLES LIKE 'pisco_%';
```

PDO se configura con excepciones, charset utf8mb4 y preparaciones nativas. No se requiere usar root como cuenta de producción. El usuario debe poder crear o alterar las estructuras previstas; si se retiran esos privilegios después del despliegue, planifique cómo habilitar las migraciones futuras.

No sustituya el prefijo pisco_ en archivos sueltos. Un cambio de prefijo requiere una modificación coordinada del código y las pruebas. Confirme la base seleccionada antes de cualquier importación o restauración.

07. Configuración privada y secretos

La ruta predeterminada es portal-private/config.php, creada a partir de config.example.php. El archivo devuelve un array PHP; no debe producir salida ni incluirse en el repositorio público.

Parámetro	Uso y control
db_*	Conexión SQL. Proteja contraseña y nombre de base junto con la cuenta del alojamiento.
install_key	Clave privada de al menos 32 caracteres para la instalación de esta revisión.
secure_cookies	true en producción HTTPS; false solo para desarrollo local HTTP.
timezone	America/Lima en esta instalación.

```
# Ejemplo de selección de un archivo privado
export PORTAL_CONFIG_FILE=/ruta/privada/config.php
```

El repositorio revisado no incluye un archivo de ajustes SMTP. Si el despliegue incorpora correo, identifique su configuración privada y las dependencias de la clave antes de trasladar el sitio.

Conserve install_key de forma privada: autoriza la instalación y debe mantenerse fuera de documentos de distribución. Antes de cambiarla en un despliegue ampliado, identifique si otros componentes dependen de ella. Genere nuevas claves con un generador criptográfico y guárdelas de manera privada.

08. Instalación inicial

La instalación crea esquema, estado inicial y cuenta administradora. Debe hacerse una sola vez sobre la base prevista, utilizando una clave de instalación privada.

1. Publique portal-private y public_html respetando la separación de directorios.

2. Cree config.php con conexión y una install_key válida.

3. Confirme HTTPS y los permisos de escritura requeridos.

4. Abra /install.php y complete clave de instalación y cuenta administradora con contraseña válida.

5. Espere el resultado y compruebe el acceso al portal y a la intranet.

6. Retire public_html/install.php una vez concluido el proceso.

7. Revise datos institucionales, áreas y remitentes antes de habilitar servicios.

PortalApp::install utiliza install.lock con flock y rechaza una instalación ya existente con HTTP 410. Aplica el esquema e inserta el estado inicial desde data/portal.json. El contenido de esa semilla corresponde a CHINCHA; no copie la carpeta privada completa desde otro sitio.

Síntoma	Comprobación
503 por configuración	Archivo privado, array PHP, conexión y longitud de clave.
403 por clave	Valor introducido y clave del archivo seleccionado.
410	El sistema ya está instalado; no elimine tablas para forzar la pantalla.

09. Servidor web, HTTPS y dominio

La dirección institucional de referencia es https://iestpchincha.edu.pe/. El proyecto espera servir desde / y utiliza fragmentos del navegador para la navegación interna, por ejemplo /#/admin.

```
DirectoryIndex index.php
Options -Indexes
<FilesMatch "(^\.|\.(env|sql|log|ini|bak)$)">
    Require all denied
</FilesMatch>
```

Estas directivas están en public_html/.htaccess. portal-private incluye Require all denied, pero debe permanecer fuera del document root. Si se utiliza un servidor distinto de Apache, reproduzca los controles con su configuración equivalente; no asuma que interpreta .htaccess.

Instale un certificado válido y configure la redirección HTTPS desde el alojamiento. Mantenga secure_cookies=true. La aplicación emite HSTS cuando detecta HTTPS o el encabezado de protocolo previsto; esa señal debe proceder de un proxy de confianza.

No se necesita una reescritura de /api/portal para los ejemplos de este manual: el punto de entrada es index.php y la ruta se pasa en el parámetro route. El repositorio de esquema 5 sirve las secciones por fragmentos; los servicios autónomos .php deben comprobarse en el despliegue de esquema 7. Las verificaciones de salud deben realizar GET: la portada rechaza métodos distintos y una consulta HEAD puede devolver 405 sin que el sitio esté caído.

10. Desarrollo local y separación de entornos

El servidor integrado de PHP sirve para trabajo local y pruebas; no sustituye al servidor de producción. Use una base y configuración propias, con SMTP externo desactivado.

```
# Desde la raíz de una copia de desarrollo
PORTAL_CONFIG_FILE=/ruta/privada/desarrollo.php \
php -S 127.0.0.1:8086 -t public_html

# Lectura del servicio, sin credenciales en la orden
curl -fsS 'http://127.0.0.1:8086/?route=api%2Fportal'
```

La configuración local debe usar secure_cookies=false únicamente si el acceso es HTTP. Restrinja la escucha a loopback cuando no necesite acceso remoto. La base no debe ser una réplica conectada directamente a producción ni reutilizar su usuario.

Entorno	Finalidad
Desarrollo	Cambios y validaciones rápidas con datos de prueba.
QA	Bases vacías y casos automatizados; restablecimiento controlado.
Preproducción	Ensayo de despliegue, migración y recuperación.
Producción	Datos institucionales y servicios reales.

Una conexión SQL correcta no valida permisos funcionales, archivos ni correo. Compruebe una operación representativa en cada área afectada. El entorno local utilizado en la revisión empleó PHP 8.3 y MariaDB 10.11; no es una afirmación sobre la versión activa del hosting institucional.

11. Modelo de persistencia

El sistema combina un documento JSON central con tablas para identidad, archivos y operación. No debe interpretarse como un esquema relacional con una tabla por cada módulo del menú.

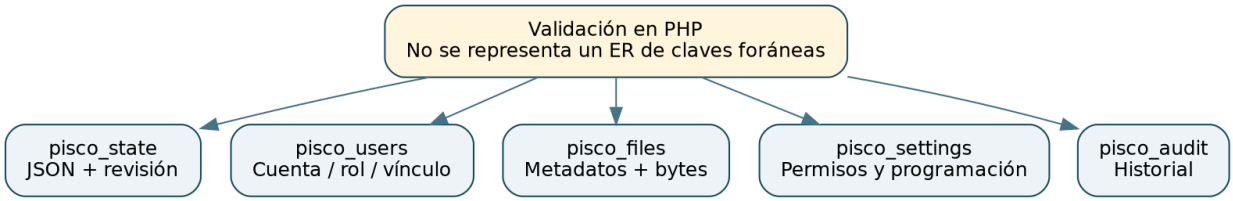


Figura 2. Relación funcional entre aplicación y tablas; no representa claves foráneas SQL.

pisco_state conserva el contenido y las colecciones del sistema en la fila id=1. La revisión permite identificar cambios del estado. Alumnos, sesiones, notas y publicaciones se almacenan en sus colecciones JSON, con reglas y referencias verificadas por PHP.

En el esquema 5 revisado existen ocho tablas físicas. Cuentas, archivos, auditoría, ajustes, límites, visitas y versión complementan pisco_state. No se declaran claves foráneas SQL; las relaciones de contenido se resuelven mediante validación de aplicación.

Para investigar un caso, empiece por el identificador funcional y el módulo. Evite actualizaciones SQL directas sobre el JSON: saltarían validación, sincronización documental, notificaciones y auditoría de negocio. Prepare cualquier corrección de datos como una intervención separada, probada y recuperable.

12. Diccionario: estado y cuentas

Las siguientes estructuras son centrales para comprender revisiones, acceso y vínculos. Los campos sensibles no deben exportarse en informes de soporte ordinarios.

Tabla / campo	Significado
pisco_state.id	Identificador de la fila de contenido; el estado principal usa 1.
pisco_state.data	JSON con configuración, contenido público y colecciones privadas.
pisco_state.revision	Contador de revisión BIGINT UNSIGNED.
pisco_state.updated_at	Marca de actualización de la fila.
pisco_users.username / email	Identificadores únicos de cuenta.
pisco_users.password	Hash generado mediante password_hash, no contraseña reversible.
pisco_users.role / profile	Rol y JSON con vínculos, área, programa o persona.
pisco_users.auth_version	Versión de autenticación para invalidar sesiones anteriores.
pisco_users.active / last_login	Habilitación y último acceso registrado.

```
-- Inventario sin contenido ni hashes de contraseña
SELECT id, username, role, active, last_login
FROM pisco_users ORDER BY username;
SELECT id, revision, updated_at FROM pisco_state;
```

En esta revisión, los fallos de acceso se registran en login_failures y locked_until de la cuenta. Cinco fallos consecutivos pueden bloquearla durante cinco minutos. El limitador de frecuencia utiliza además pisco_rate_limits. Compruebe ambos mecanismos según el mensaje observado.

13. Diccionario: archivos y operación

Las tablas operativas complementan al estado JSON y tienen ciclos de retención distintos. Antes de borrar datos identifique el procedimiento de mantenimiento de cada componente.

Tabla	Datos y uso
pisco_files	id, propietario, sesión, nombre, MIME, tamaño, SHA-256, LONGBLOB y fecha.
pisco_audit	Fecha, actor, rol, acción, entidad, clave de registro y detalle JSON.
pisco_settings	Clave de ajuste y valor JSON; incluye permisos de roles y programación de copias.
pisco_rate_limits	Clave hash, contador y caducidad de límites.
pisco_schema	Versión aplicada del esquema.
pisco_visits	Conteo agregado de vistas por día.

```
-- Solo metadatos agregados
SELECT COUNT(*) AS archivos, SUM(size) AS bytes
FROM pisco_files;
SELECT version FROM pisco_schema WHERE id=1;
SELECT COUNT(*) AS cuentas FROM pisco_users;
```

No confunda tamaño total de archivos con espacio total de base: índices, auditoría, JSON y sobrecarga del motor también consumen capacidad. Los destinatarios, cuerpos de correo y documentos pueden contener datos personales. Para soporte prefiera conteos y registros concretos, no volcados completos.

14. Estado JSON y contratos funcionales

Las colecciones institucionales se validan antes de confirmarse. Los contratos de referencia y las clases de dominio describen estructura, identidades y relaciones que la interfaz debe respetar.

Componente	Tratamiento
config y contenido público	Datos institucionales y colecciones editoriales.
academico	Maestros, alumnos, sesiones, publicaciones y documentación académica.
Colecciones privadas	Mensajes, reclamos, expedientes, cuentas y otros registros con proyección autorizada.
contracts/*.json	Referencias estructurales utilizadas por la base funcional.
Rules / Domain / Academic	Validación de forma, identidad y relaciones de negocio.

PortalPolicy proyecta lo que una cuenta puede ver y modificar. La respuesta pública no debe equipararse con el estado completo guardado en pisco_state. Un desarrollador no debe reconstruir el estado total desde una vista parcial y sobrescribirlo como si fuese completo.

Al agregar una colección o un campo revise validación, proyección pública, permisos, clave estable, vínculos, exportación y restauración. Actualice pruebas que cubran datos inválidos y roles restringidos, además del caso exitoso.

Conserve la distinción entre contenido inicial y datos instalados: editar data/portal.json afecta nuevas instalaciones; no debe suponerse que actualiza los registros ya guardados en una base existente.

15. Versiones y compatibilidad de esquema

La revisión 14e4488271c3 de ranyuo/chincha declara PortalSchema::VERSION = 5. El servicio del sitio publicado informó schema_version = 7 el 10 de noviembre de 2026. Son referencias diferentes y no deben tratarse como un mismo despliegue.

Referencia	Valor comprobado
Código disponible	Esquema 5, ocho tablas y 44 columnas en schema.sql.
Instalación local	PHP 8.3 y MariaDB 10.11, creada desde ese código.
Sitio publicado	Respuesta del servicio api/portal con esquema 7.
Base del alojamiento	Sin acceso SQL directo; estructura física pendiente de contraste.

```
-- Ejecutar con acceso SQL autorizado al entorno adecuado
SELECT version FROM pisco_schema WHERE id=1;
SHOW TABLES;
SHOW CREATE TABLE pisco_state;
```

1. Obtenga el conjunto de archivos y la revisión realmente desplegados antes de una actualización.

2. Guarde una copia consistente de base, archivos y configuración.

3. Compare esquema de origen y destino en un entorno aislado.

4. Verifique lectura, edición, permisos, documentos y recuperación.

5. Publique únicamente una versión compatible y registre el resultado.

No sustituya el sitio publicado por el repositorio de esquema 5 como si se tratase de una actualización. Un retorno de 7 a 5 necesita un plan de compatibilidad y recuperación. Cambiar el valor de pisco_schema no ejecuta una migración.

El diccionario adjunto documenta el DDL comprobado del repositorio. Para certificar el esquema del hosting, soporte debe proporcionar el DDL de esa instalación, sin datos personales ni credenciales.

16. Entrada HTTP y enrutamiento

index.php distingue la portada y las rutas JSON. El método y la ruta se validan en el servidor; la navegación con # pertenece al navegador.



Figura 3. Recorrido de una petición en la aplicación.

```
# Portada: GET, no HEAD
curl -fsS https://iestpchincha.edu.pe/ -o portada.html

# API por parámetro route
curl -fsS \
 'https://iestpchincha.edu.pe/?route=api%2Fportal' \
 -o portal-publico.json
```

En el código revisado, las secciones públicas utilizan fragmentos como /#/servicios/mesa-de-partes. No existen archivos independientes mesa-de-partes.php, libro-de-reclamaciones.php, bolsa-laboral.php ni certificados.php en esa revisión. Su disponibilidad debe comprobarse en el despliegue publicado.

Las respuestas JSON se codifican con opciones que protegen su inserción y normalizan los datos. Los errores esperados incluyen código HTTP y message; las excepciones generales se registran de forma limitada y presentan un mensaje de servicio no disponible.

17. Sesión, CSRF y autenticación

La sesión PHP utiliza PISCOSESSID, cookies HttpOnly y SameSite=Lax; Secure depende de secure_cookies. El control de inactividad previsto es de 1800 segundos.

Control	Comportamiento revisado
Ingreso	password_verify; renovación de ID de sesión y token CSRF al autenticar.
CSRF	X-CSRF-TOKEN en rutas API que no son GET. Formularios auxiliares usan _csrf.
Revocación	auth_version de sesión debe coincidir con la cuenta.
Inactividad	Se limpia la sesión y se regenera el identificador al superar el plazo.
Contraseña temporal	Se restringen operaciones hasta completar el cambio.

En el código de referencia, login_failures y locked_until controlan los fallos por cuenta. A partir de cinco fallos puede exigirse esperar cinco minutos; existe además un límite de frecuencia de ingreso. El comportamiento del despliegue debe contrastarse cuando se utilice otro esquema.

Una respuesta 419 requiere recargar el contexto de sesión y obtener el token vigente. No desactive CSRF ni reutilice tokens de otra cuenta para resolver una integración. Las solicitudes administrativas deben usar una sesión legítima, no una contraseña escrita en una URL.

Los controles se verifican en PHP. Ocultar opciones en JavaScript o manipular el menú no cambia la autorización del servidor.

18. API de contenido y administración

La API no es un reemplazo público de todas las operaciones administrativas. La sesión y el rol determinan la proyección y las acciones permitidas.

Método y route	Función
GET api/portal	Carga de datos visibles, revisión, usuario, CSRF, año y versión de esquema.
PUT api/portal	Guardado validado de cambios, con control de revisión y permisos.
POST / DELETE api/session	Ingreso / cierre de sesión.
POST api/password	Cambio de contraseña de la cuenta.
POST api/cuentas	Operaciones autorizadas de cuentas.
GET / PUT api/smtp	Función de la versión publicada; no incluida en el código de esquema 5.
GET api/notificaciones	Función de la versión publicada; no incluida en el código de esquema 5.

Las rutas anteriores se invocan como ?route=api%2Fportal y equivalentes. Antes de enviar una modificación cargue el estado visible y el token de la misma sesión. No incluya hashes de contraseña, secretos o una colección privada tomada de otra cuenta.

La respuesta de guardado devuelve revisión y datos normalizados autorizados. El cliente debe incorporar esa respuesta como nueva base de edición. Una respuesta 409 exige recargar y resolver el conflicto; repetir ciegamente el mismo cuerpo no es una estrategia de recuperación.

Consulte el anexo de rutas para los servicios ciudadanos y los archivos auxiliares que usan formularios HTTP en lugar de JSON.

19. Formularios públicos e idempotencia

La recepción valida tipo de solicitud, estructura, prueba antispam y una clave de idempotencia con forma UUID. Las referencias de adjuntos se validan durante el trámite; compruebe el tratamiento de los archivos no vinculados antes de definir una rutina de limpieza.

1. Solicitar el desafío apropiado y conservar la sesión del navegador.

2. Completar los datos y declaraciones previstos por el formulario.

3. Preparar una idempotency_key para el intento lógico.

4. Enviar record y proof a api/solicitudes/<tipo>.

5. Ante reintento de la misma operación, conservar clave y contexto de sesión.

6. Confirmar el identificador y cargo antes de iniciar otro trámite.

El identificador interno combina sesión, tipo e idempotency_key mediante SHA-256. Una repetición reconocida devuelve el registro existente con estado HTTP 200; un registro nuevo devuelve 201. Esta protección está vinculada al contexto de la sesión y no constituye una unicidad documental global entre navegadores distintos.

El servidor aplica cuota de solicitudes y valida referencias de archivos, total de bytes, destino y numeración. La reutilización reconocida evita almacenar de nuevo los adjuntos o consumir otra cuota de subida por el mismo archivo del reintento.

Para diagnosticar un envío interrumpido revise registro y cargo antes de borrar cookies o abrir una nueva sesión. La verificación de certificados por DNI también exige la prueba del servicio; código y QR tienen su recorrido de consulta propio.

20. Adjuntos, validación y descarga

Los adjuntos se guardan como bytes en pisco_files, con SHA-256, tamaño, MIME y propiedad. No se trata de archivos arbitrarios publicados por su nombre original en una carpeta web.

Control	Implementación y límite
Nombre	Normalización de basename, retirada de controles y longitud válida.
Contenido	MIME detectado por fileinfo y extensión permitida para ese MIME.
Tamaño global	Hasta 40 MiB por archivo en la capa de almacenamiento.
Solicitud ciudadana	Límites por archivo, formato y cantidad definidos en el servicio. No confundirlos con un único máximo total.
Reclamaciones	Máximo anunciado y aplicado de 10 MiB en los formatos admitidos.
Entrada JSON	Rechaza CONTENT_LENGTH superior a 64 MiB; no equivale al límite de subida de PHP.

La descarga comprueba propiedad o presencia autorizada en la proyección visible. La respuesta indica MIME, tamaño y Content-Disposition; para contenido servido se añade una política restrictiva de contenido. No elimine el control de acceso porque un enlace fileId parezca aleatorio.

Coordine upload_max_filesize, post_max_size, límites del servidor web, memoria y max_allowed_packet con el uso admitido. El menor de esos límites puede rechazar una petición antes de que PHP ejecute sus reglas.

La comprobación MIME y extensión no demuestra que el documento carezca de contenido malicioso. Si se necesita análisis antimalware, debe implantarse y verificarse como un control adicional, sin atribuírselo al código actual.

21. Concurrencia, revisiones y parches

Las operaciones que cambian estado emplean transacciones y bloqueo de la fila principal cuando procede. Un fallo debe investigarse comprobando el estado persistido antes de repetir la solicitud.

Tipo de cambio	Regla de conflicto
value	Compara before con el valor actual y aplica after.
rows	Compara cada fila por una clave estable; controla altas, cambios y bajas.
object	Compara cada propiedad antes de actualizar o retirar un valor.
Guardado completo	Exige coincidencia de revision; no debe construirse desde datos ajenos.

PortalPatch limita rutas y cantidad de cambios. Solo se admite el anidamiento previsto bajo academico. Las identidades se validan y una modificación concurrente produce 409 en lugar de sustituir silenciosamente lo ya guardado.

Después de guardar, use la revisión y normalized de la respuesta para restablecer la base del editor. Si recibe conflicto, recargue y compare el registro; evite convertir automáticamente una edición parcial en un reemplazo completo del JSON.

La fila de estado común concentra modificaciones de varias áreas. En instalaciones con mayor concurrencia mida tiempos de bloqueo, tamaño de JSON y frecuencia de operaciones antes de decidir una refactorización. Una optimización debe conservar las reglas transaccionales, auditoría y sincronización de notificaciones.

22. Permisos, cuentas y alcance

PortalPolicy combina rol, módulos habilitados, niveles académicos y vínculos personales. La autorización no se reduce a que exista una cuenta activa.

Ámbito	Comprobación técnica
Coordinador	Programa asignado y correspondencia en progMap.
Alumno / egresado	Vínculo est:<id> con alumno existente.
Docente	Carga docente y registro de persona coherentes.
Empresa	Vínculo emp:<id> y acceso a sus ofertas/postulaciones.
Área receptora	Cuenta del área y expedientes derivados dentro de su alcance.
Dirección	Consulta en áreas donde el rol no debe ejecutar atención.

La edición de roles se almacena en pisco_settings y usa un valor expected para evitar sobreescritura concurrente. El administrador está protegido y las personalizaciones no pueden ampliar el rol a módulos fuera de su ámbito institucional previsto.

Al cambiar permisos se incrementa auth_version para invalidar sesiones afectadas. La creación de cuentas valida usuario, correo, rol y vínculos; la contraseña requiere al menos 12 caracteres y no más de 72 bytes.

No corrija un problema de vinculación asignando el rol administrador. Distinga credenciales, habilitación de cuenta, permiso de módulo y selección de datos. Cada causa requiere una intervención diferente y trazable.

23. Integridad académica y documentos

El módulo académico relaciona programas, unidades, docentes, alumnos y emisiones. Una corrección de datos maestros debe conservar la trazabilidad de los documentos ya entregados.

Proceso	Regla que debe conservarse
Maestros	Programa → módulo formativo → unidad didáctica; asignaciones y ambientes coherentes.
Horarios	Grupo y periodo, validación de cruces y publicación para consulta.
Notas	Unidad, alumno y periodo; normalización para evitar duplicados.
Certificación y títulos	Registro de origen y sincronización de verificación documental.
Documento emitido	Protección de titular y datos históricos; anulación y nueva emisión para correcciones.

En el repositorio de esquema 5, Academic.php y las reglas del dominio validan las colecciones académicas. No incluye una clase AcademicDocuments.php. Antes de trasladar o corregir emisiones del sitio de esquema 7, identifique la implementación desplegada y compruebe sus reglas de historial en un entorno aislado.

Los estudiantes ven el horario publicado de su grupo en el periodo vigente. Una cuenta que inicia sesión sin mostrar el horario puede tener un vínculo o grupo incorrecto, una publicación ausente o un periodo distinto; no es necesariamente una falla del motor SQL.

Al modificar validaciones académicas incluya pruebas de identidades inválidas, periodos históricos, documentos anulados y datos de programas ajenos, además de la operación normal del rol administrador.

24. Correo notificador y configuración

La versión publicada informa esquema 7 y puede incorporar correo notificador. El repositorio de Chincha disponible declara esquema 5 y no incluye el componente SMTP. Esta sección proporciona un procedimiento de comprobación para soporte, sin atribuirle una implementación no inspeccionada.

Inventario que debe completar soporte

Dato	Comprobación
Disponibilidad	Confirmar Configuración y permisos de administración.
Servidor y puerto	Usar los valores aprobados por el proveedor de correo.
Cifrado	Distinguir STARTTLS de SSL/TLS y validar el certificado.
Cuenta y remitente	Confirmar autenticación, remitente autorizado y políticas del dominio.
Almacenamiento	Identificar archivo o servicio privado; nunca incluir secretos en un manual.

1. Identifique la versión desplegada y sus instrucciones de correo.

2. Revise la configuración sin revelar ni reemplazar la contraseña existente.

3. Ensaye el envío en un entorno controlado con un destinatario autorizado.

4. Compruebe aceptación del servidor y recepción final por separado.

5. Registre fecha, resultado y mensaje de error sin credenciales.

Guardar ajustes y activar envíos son acciones diferentes. Si existe una cola pendiente, activar el servicio puede despachar avisos anteriores. Revise su alcance antes del cambio.

No se certifica para Chincha un algoritmo de cifrado de la clave SMTP ni una ruta de archivo privada: esa información requiere inspección de los componentes desplegados.

25. Notificaciones y seguimiento de entrega

El registro de un trámite y la entrega de su aviso son hechos distintos. Verifique primero el expediente o reclamo para evitar duplicarlo cuando el usuario no reciba correo.

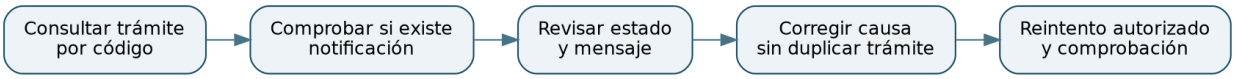


Figura 4. Flujo operativo de comprobación de una notificación.

Situación	Acción de soporte
Pendiente	Comprobar que el envío está habilitado y que existe un proceso de entrega.
Error	Revisar mensaje, destinatario y configuración antes de reintentar.
Enviado	Confirmar qué significa ese estado en la versión desplegada; no equivale a lectura.
Sin registro del aviso	Verificar si el evento genera correo en esa versión y configuración.

1. Localice el trámite por su código, sin volver a crearlo.

2. Identifique el evento de notificación y el destinatario previsto.

3. Revise el estado y la hora con el personal autorizado.

4. Corrija la causa antes de un reintento controlado.

5. Conserve evidencia del resultado y confirme que no se duplicó el trámite.

El código disponible de esquema 5 no declara pisco_mail. Los campos, número máximo de intentos, frecuencia, retención y reglas de cancelación del sitio de esquema 7 quedan pendientes de contraste con su código o DDL. No ejecute consultas sobre una tabla solo porque aparezca en otro sistema.

Solicite para el inventario técnico únicamente estructura y estadísticas agregadas. Los destinatarios y cuerpos de correo pueden contener información privada.

26. Cron y tareas de mantenimiento

El repositorio revisado incluye portal-private/cron.php para respaldos programados. Su ejecución debe utilizar PHP CLI y una configuración privada del mismo sitio.

```
# Ejemplo de tarea diaria; sustituya binario y rutas
0 2 * * * /ruta/php /ruta/portal-private/cron.php
```

1. Compruebe la ruta del PHP CLI y sus extensiones.

2. Confirme que la configuración resuelve a la base del sitio correcto.

3. Active y configure el respaldo desde la intranet, con hora y retención apropiadas.

4. Ejecute una prueba autorizada y revise la salida.

5. Compruebe la existencia, fecha y legibilidad de la copia generada.

La clase Backups aplica un bloqueo de archivo al programador, comprueba hora y última ejecución y conserva la cantidad configurada de copias programadas. La entrada de cron debe ejecutarse en un horario compatible.

En el código revisado, cron.php entrega un mensaje de copia completada o de ausencia de pendientes y termina con error si no puede completar. Capture su salida en un registro privado con rotación.

No programe mail-worker.php con esta revisión: ese archivo no está incluido. Si el despliegue de esquema 7 lo incorpora, obtenga sus instrucciones y compruebe su comportamiento antes de crear otra tarea.

27. Respaldo: alcance y protección

La copia portable .piscobak está diseñada para recuperar contenido, cuentas, vistas agregadas y adjuntos. No equivale a una imagen completa del hosting.

Elemento	Cobertura de la copia portable
pisco_state	Estado y revisión.
pisco_users	Cuentas y perfiles; contiene hashes sensibles.
pisco_visits	Agregados de visitas.
pisco_files	Metadatos y bytes en fragmentos.
Permisos de roles	Incluidos en la cabecera.
Configuración y SMTP cifrado	Debe conservarse por separado; SMTP solo si el despliegue lo incorpora.
Cola, auditoría y otros ajustes	No se restauran como parte de las cuatro tablas de contenido.

El archivo utiliza compresión gzip, registros JSON por línea, conteos y hash SHA-256. Estos mecanismos verifican consistencia, pero no cifran el respaldo ni prueban por sí solos la autenticidad de su procedencia. Proteja su transporte y almacenamiento.

Los respaldos programados se conservan según la retención configurada, entre 2 y 30 copias, por defecto 7. Este número no representa días garantizados si las ejecuciones no se realizan. Mantenga también una copia privada fuera del alojamiento y pruebe la recuperación.

Defina con la institución la pérdida máxima de datos admisible y el tiempo objetivo de recuperación. El código no establece por sí solo un RPO o RTO contractual.

28. Restauración portable

La recuperación sustituye contenido y cuentas. Requiere autorización, una ventana acordada y una copia previa verificable del estado que se va a reemplazar.



Figura 5. Controles principales de una restauración portable.

1. Identifique archivo, fecha, procedencia y compatibilidad de esquema.

2. Conserve configuración privada, install_key y ajuste SMTP por separado.

3. Desde Respaldos seleccione la copia o cargue el archivo admitido.

4. Introduzca la contraseña actual y la confirmación RESTAURAR requeridas.

5. Espere validación y confirmación antes de abrir nuevas operaciones.

6. Vuelva a ingresar y compruebe cuentas, contenido, archivos y servicios.

La interfaz crea una copia previa del estado actual. La clase valida formato, versión, conteos, final de archivo, datos estructurales, hashes de adjuntos y existencia de un administrador activo. Ante inconsistencias revierte la transacción.

Después de restaurar, vuelva a autenticar y compruebe revisión, cuentas, contenido y adjuntos. El código y la base de destino deben aceptar el esquema de la copia. No intente restaurar una copia de esquema 7 con el programa de esquema 5. La cobertura de componentes adicionales del sitio publicado debe comprobarse por separado.

29. Exportación SQL y traslado de hosting

backup.php genera un volcado autenticado por POST y CSRF. En el repositorio revisado exporta las ocho tablas SQL y sus datos. La cobertura de un despliegue de esquema 7 debe contrastarse con su propio exportador.

Componente del traslado	Acción
Archivos de aplicación	Copiar la versión completa con la raíz pública correcta.
Base de datos	Elegir exportación SQL o restauración portable según el escenario.
config.php	Ajustar la conexión del nuevo hosting y conservar secretos fuera de la web.
SMTP protegido	Solo si está incorporado en la versión desplegada; identificar configuración y claves.
Cron y HTTPS	Recrear las tareas y verificar certificado y cookies.

El SQL contiene cuentas y adjuntos; los binarios grandes se emiten en fragmentos. No lo adjunte a tickets públicos ni lo deje descargable en public_html. Su importación es una operación de base de datos, no una carga del archivo en el restaurador .piscobak.

Pruebe el destino antes de cambiar DNS: acceso, portal, consulta documental y archivos representativos. Mantenga el envío de correo controlado para no duplicar avisos entre origen y destino.

Al importar SQL se recupera una cobertura distinta de la copia portable, incluida información operativa como auditoría y límites. Evalúe versión y datos transitorios antes de reabrir. No publique dos instalaciones con la misma base como un mecanismo improvisado de transición.

30. Actualización y recuperación de una versión

Una actualización debe mantener compatible el conjunto de código y esquema y preservar configuración, respaldos y archivos generados que no pertenecen al paquete de código.

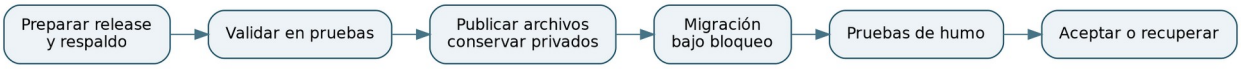


Figura 6. Secuencia recomendada de publicación y validación.

1. Identifique la versión y obtenga un respaldo de datos más la configuración privada.

2. Ensaye las migraciones y las funciones afectadas en una copia aislada.

3. Publique la nueva versión sin borrar las carpetas que conservan datos privados.

4. Retire install.php si el paquete lo incluye y la aplicación ya está instalada.

5. Compruebe esquema, ingreso, consulta, adjuntos y tareas programadas.

6. Registre aceptación o active el procedimiento de recuperación acordado.

Para recuperar una versión, no basta con copiar PHP anterior si el esquema o los datos ya cambiaron. Identifique un punto coherente de código, base y configuración. La restauración puede descartar operaciones posteriores al respaldo; preserve evidencia y coordine la ventana antes de ejecutarla.

No interprete una página que carga como una validación completa del despliegue. Un servicio puede fallar por permisos, límites de subida, cifrado SMTP, sesiones o tareas cron aunque la portada pública responda.

31. Auditoría, errores y observabilidad

El historial funcional y los registros del servidor responden preguntas distintas. La auditoría identifica acciones de usuarios y operaciones; el registro PHP permite investigar fallos de infraestructura o excepciones.

Fuente	Uso
auditoria.php	Filtrado por texto y fechas, paginación y exportación CSV de resultados.
pisco_audit	Eventos con actor, entidad y clave de registro.
Registro PHP del hosting	Clases/códigos de error y fallos de tareas.
Salida cron	Resultado de cada ejecución programada.
Cola de correo	Estado, intentos y próxima entrega, dentro del alcance autorizado.

El historial de auditoría contiene acciones y cambios. Confirme con el responsable la política de conservación y el proceso que la aplica en la versión desplegada; no suponga una depuración automática solo por la existencia de un cron de respaldos.

```
-- Resumen para diagnóstico sin volcar detalles
SELECT action, COUNT(*) AS total
FROM pisco_audit
WHERE occurred_at >= CURRENT_DATE
GROUP BY action ORDER BY total DESC;
```

Al reportar una incidencia registre la hora con su zona horaria, módulo, código de registro y mensaje exacto. Correlacione la hora del servidor, PHP, base y navegador antes de ordenar los eventos.

32. Capacidad y rendimiento

La capacidad depende del tamaño del estado, los binarios, la concurrencia y el alojamiento. Las cifras de las pruebas locales no deben presentarse como límites garantizados del servicio publicado.

Componente	Indicador recomendado
Estado JSON	Bytes, duración de carga/guardado y tamaño de la respuesta.
Concurrencia	Tiempos de bloqueo de fila y errores 409.
Adjuntos	Conteo, bytes y espacio de base; límites de PHP y MySQL.
Correo	Pendientes vencidos, fallos y tasa efectiva de procesamiento.
Respaldos	Duración, tamaño, espacio libre y antigüedad de la última copia.
Auditoría	Crecimiento, consulta y cumplimiento de retención.

```
SELECT OCTET_LENGTH(data) AS bytes_estado
FROM pisco_state WHERE id=1;
SELECT COUNT(*) AS total, SUM(size) AS bytes
FROM pisco_files;
```

Los adjuntos en LONGBLOB aumentan el consumo de la base. No confunda el límite del formulario con la capacidad de una petición, memoria PHP o max_allowed_packet. Mida una subida representativa en preproducción.

El cliente versiona recursos estáticos mediante parámetros de modificación y los archivos de caché por contenido pueden tener retención larga. No aplique caché pública indiscriminada a API o páginas con sesión. Evalúe cambios de infraestructura con pruebas funcionales y de concurrencia.

33. Controles de seguridad y mantenimiento

La operación segura combina controles de aplicación, alojamiento y procedimientos. Los resultados de QA comprueban casos concretos y no sustituyen una revisión de seguridad completa.

Control	Responsabilidad de operación
Secretos	Configuración fuera del document root y acceso privado al respaldo de claves.
TLS y sesión	Certificado válido, cookies seguras y proxy confiable.
Acceso	Cuentas individuales, roles mínimos y vínculos correctos.
Contenido	Validación del servidor, archivos admitidos y publicación autorizada.
Dependencias	Integridad y avisos de bibliotecas incluidas.
Recuperación	Copias protegidas y restauración probada.

No use permisos 777 como solución general ni desactive verificación TLS, hashes o validaciones para superar un error. Ajuste propietario y permisos de acuerdo con el usuario que ejecuta PHP.

Revise las bibliotecas de assets/vendor, sus cabeceras y licencias y las referencias en FUENTES.md. El repositorio de Chincha no incluye versions.json ni PHPMailer. Antes de incorporar una actualización compruebe origen, compatibilidad y comportamiento en pruebas.

Ante una sospecha de compromiso preserve registros y copias, limite las operaciones afectadas y coordine la recuperación con el responsable del alojamiento. Registre la versión exacta y los componentes privados que deban conservarse.

34. Comprobaciones y pruebas de aceptación

El repositorio publicado de Chincha no incluye el directorio tests ni un flujo de integración continua. La aceptación requiere comprobaciones reproducibles que se ejecuten en una instalación aislada.

```
# Desde una copia de pruebas con PHP disponible
find portal-private public_html -name "*.php" -type f \
 -exec php -l {} \;

curl -fsS http://127.0.0.1:8086/ -o portada.html
```

Comprobación	Criterio de aceptación
Sintaxis	Todos los archivos PHP analizados terminan sin error.
Instalación	La base local se crea y carga el portal.
Autenticación	La cuenta de prueba accede al panel y puede cerrar sesión.
Permisos	Una consulta sin sesión no expone colecciones privadas.
Navegación	Las rutas de los talleres abren los módulos esperados.
Estructura	Las tablas y columnas locales coinciden con el DDL revisado.

La validación sintáctica no prueba funcionamiento ni seguridad. Complete casos representativos de edición, concurrencia, adjuntos y recuperación cuando disponga de un entorno desechable y datos de prueba.

No presente resultados de estabilidad, correo o volumen de otro instituto como evidencia de Chincha. El anexo C describe solamente comprobaciones realizadas para esta entrega.

Conserve fecha, versión del código, comando y resultado. Antes de automatizar publicaciones, prepare un ejecutor que falle cuando una comprobación no se ejecute o no cumpla su criterio.

35. Diagnóstico por síntomas

Investigue desde el mensaje y la capa que lo produce. Antes de repetir un cambio confirme si ya se guardó; antes de recuperar una copia preserve el estado actual.

Síntoma	Orden de comprobación
503 en portada	Configuración privada → conexión → tablas → versión y permisos de migración.
419	Sesión/cookie → token vigente → recarga del formulario.
409	Datos concurrentes → recarga → reaplicación del cambio autorizado.
413 o carga rechazada	Servidor web → PHP → total JSON/adjuntos → reglas del servicio.
403	Cuenta y rol → vínculo → alcance por área o programa → estado documental.
423 al ingresar	Fallos de cuenta y límite de frecuencia → espera prevista → procedimiento de desbloqueo.
Avisos pendientes	SMTP habilitado → credencial → cron → estado y próxima ejecución.
Sin horario	Alumno/cuenta → grupo → periodo → publicación.

Una respuesta 405 a HEAD no equivale a caída: pruebe GET sobre la portada o API. Si el navegador y PHP muestran comportamientos distintos, capture ruta, método, estado HTTP y hora antes de modificar configuración.

Los fallos de correo no justifican reenviar automáticamente trámites. Los conflictos de revisión no se resuelven alterando el contador SQL. Los errores de respaldo no se solucionan quitando la validación de integridad. Escale con una evidencia mínima y sin credenciales.

36. Rutinas de operación y continuidad

Asigne responsables y documente la ejecución de las tareas. Una casilla de cron configurada o un archivo de respaldo existente no demuestran por sí solos que el servicio sea recuperable.

Frecuencia orientativa	Actividad
Diaria	Disponibilidad por GET, cola, errores relevantes y última copia.
Semanal	Espacio de base y archivos, tareas cron y cuentas pendientes.
Mensual	Prueba de restauración aislada, permisos y documentación vigente.
Por cambio	Versión, respaldo, migración, pruebas y decisión de aceptación.
Por baja o incidente	Revocación de acceso y conservación de evidencia según el procedimiento.

La frecuencia se ajusta al volumen y a los objetivos institucionales. Registre fecha de la última copia útil, tiempo de recuperación observado y alcance restaurado. Una copia que nunca se probó no debe considerarse una garantía de continuidad.

Mantenga un inventario privado con dominio, rutas, versiones, base, remitente, responsables y ubicación de secretos. Este manual aporta la estructura del procedimiento; no reemplaza ese inventario operativo.

Al cerrar un mantenimiento, deje constancia de lo que cambió, pruebas realizadas, limitaciones conocidas y forma de recuperación. Conserve el manual técnico separado del manual de usuario y del cuaderno de talleres para que cada perfil consulte la información que necesita.

Anexo A. Configuración y comprobaciones

Plantilla orientativa de config.php. Sustituya base y usuario y complete contraseña y clave de instalación de forma privada. Los campos vacíos son intencionales: esta plantilla no está lista para una instalación real.

```
<?php
return [
    'db_host' => 'localhost',
    'db_port' => 3306,
    'db_name' => 'BASE_ASIGNADA',
    'db_user' => 'USUARIO_ASIGNADO',
    'db_password' => '',
    'install_key' => '',
    'secure_cookies' => true,
    'timezone' => 'America/Lima',
];
```

Comprobaciones de preparación

```
# Desde la raíz de la copia; no imprime secretos
php -l portal-private/config.php
php -r 'foreach (["pdo_mysql","fileinfo","openssl",
 "zlib","json","session"] as $e) {
 echo $e,": ",extension_loaded($e)?"OK":"FALTA",PHP_EOL;
}'
```

La sintaxis válida no comprueba conexión, permisos ni comportamiento. Complete también acceso a base, consulta HTTP y una prueba representativa del servicio en el entorno adecuado.

Rutas que debe identificar

Dato	Valor que completa soporte
Raíz pública / directorio privado	_____
Archivo de configuración	_____
Binario PHP CLI	_____
Registro de errores y copias	_____

Anexo B. Rutas y respuestas HTTP

Los nombres api/... se pasan mediante el parámetro route de index.php. Los métodos distintos de GET requieren el token de la sesión; las autorizaciones específicas se verifican en cada operación.

Método	route o archivo	Objeto
GET	api/portal	Carga visible y revisión.
POST / DELETE	api/session	Ingreso / salida.
POST	api/password	Cambio de clave.
PUT	api/portal	Guardado validado.
POST	api/cuentas	Operaciones de cuenta.
POST	api/desafio	Prueba del servicio.
POST	api/solicitudes/<tipo>	Recepción ciudadana.
POST	api/consultas/<tipo>	Seguimiento o verificación.
POST / GET	api/archivos / api/archivo	Subida / descarga autorizada.
POST	api/empresas	Registro empresarial.
GET / PUT	api/smtp	Comprobar en esquema 7; no incluido en el código de esquema 5.
GET	api/notificaciones	Comprobar en esquema 7; no incluido en el código de esquema 5.
POST	api/notificaciones/reintentar	Comprobar en esquema 7; no incluido en el código de esquema 5.
POST	api/eventos / api/visitas	Eventos y visitas; rutas presentes en el esquema 5.
POST	backup.php	Volcado SQL autenticado.
GET / POST	respaldos.php	Gestión de copias.
GET	auditoria.php	Historial y CSV.
GET / POST	accounts.php / roles.php	Administración auxiliar.

Estados a interpretar

200: lectura o repetición reconocida. 201: creación. 403: autorización o clave rechazada. 404: ruta o registro. 405: método. 409: conflicto. 410: instalación ya existente. 413: tamaño. 419: CSRF/sesión. 422: validación. 423: bloqueo de acceso. 429: límite. 503: configuración, migración o servicio no disponible. Lea el mensaje concreto y el contexto; el código no identifica por sí solo la causa raíz.

Anexo C. Evidencia de pruebas

Comprobaciones realizadas en la copia local de ranyuo/chincha 14e4488271c3, con PHP 8.3 y MariaDB 10.11. La lectura del sitio publicado y el ingreso se comprobaron por separado.

Comprobación	Resultado observado
Sintaxis PHP	25 archivos sin errores.
Instalación local	Creada desde el código de esquema 5.
Tablas y columnas	8 tablas y 44 columnas verificadas en la base local.
Navegación de talleres	24 recorridos comprobados.
Ingreso local	Sesión administrativa confirmada.
Sitio publicado	HTTP 200; ingreso confirmado; esquema informado 7.
Contraste de versiones	Repositorio 5 frente a sitio 7; requiere código y DDL del despliegue para equipararlos.

Alcance de los resultados

Las capturas corresponden a la instalación local. La consulta y el ingreso en el sitio publicado no demuestran equivalencia de código, estructura física, configuración SMTP ni rendimiento. No se crearon trámites ni se publicaron contenidos en producción para esta entrega.

Validaciones pendientes antes de una intervención

- Obtener el código y DDL del despliegue de esquema 7 y compararlos con la referencia.
- Preparar casos aislados de correo, concurrencia, adjuntos y recuperación.
- Comprobar una recuperación completa en un entorno de pruebas autorizado.
- Medir capacidad y tiempos con una carga representativa antes de comprometer niveles de servicio.

Esta edición no atribuye a Chincha resultados de suites de estabilidad, correo o volumen de otros institutos. Conserve la evidencia de cada intervención junto a su revisión y fecha.

Anexo D. Inventario y control de entrega

Inventario de bibliotecas del navegador

Biblioteca	Versión	Referencia
jsPDF	2.5.1	jspdf.umd.min.js
AutoTable	3.8.2	jspdf.plugin.autotable.min.js
SheetJS CE	0.20.3	xlsx.full.min.js
QR generator	1.4.4	qrcode.min.js

Las referencias de origen y las licencias están en public_html/assets/vendor/FUENTES.md y los archivos LICENSE. Esta revisión de Chincha no incluye versions.json ni PHPMailer. No confunda este inventario con un análisis de vulnerabilidades actualizado.

Lista de aceptación técnica

- Dominio HTTPS, raíz pública y directorio privado correctos.
- PHP web y CLI, extensiones y conexión a base confirmados.
- Esquema y cuentas correctos; instalador retirado.
- Consulta, permiso restringido y archivo representativo comprobados.
- SMTP y cron revisados sin duplicar avisos ni exponer secretos.
- Copia protegida y recuperación ensayada en un entorno aislado.
- Versión, evidencia y procedimiento de recuperación registrados.

Fuentes técnicas de la edición

README.md; config.example.php; schema.sql y Schema.php; Portal.php y bootstrap.php; Policy.php, Accounts.php, Files.php, Academic.php, Audit.php y Backups.php; public_html/index.php; y assets/vendor/FUENTES.md del repositorio ranyuo/chincha, revisión 14e4488271c3.

Este manual debe actualizarse cuando cambien rutas, estructura, esquema, reglas de seguridad o procedimientos de recuperación. Los responsables, rutas del alojamiento y referencias de secretos se conservan en un inventario privado separado.